Southern New Hampshire University

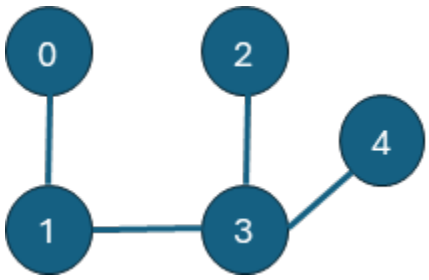
CS 505 Module Six Activity Template

Complete this template by replacing the bracketed text with the relevant information.

Part One: Identifying Trees

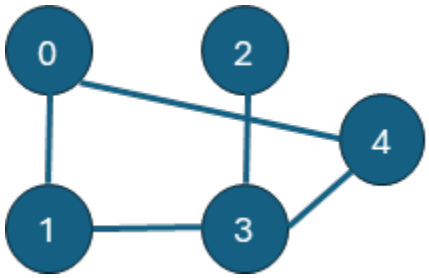
For Part One, determine whether each graph provided is a tree.

1. Explain whether each graph provided is a tree. If the provided graph is a tree, identify the root, leaves, and internal nodes. If the provided graph is not a tree, explain why.



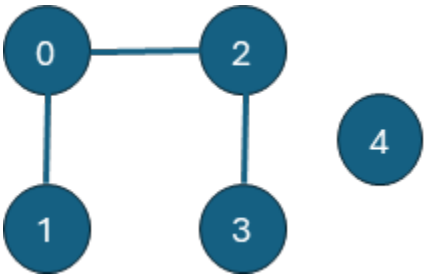
Answer:

This graph is a tree because it is connected and acyclic.
The internal vertices are 1 and 3 because they each have more than one connection.
The leaf vertices are 0, 2, and 4 because each has degree 1.
Since this is an unrooted tree, any vertex could be selected as the root.



Answer:

This graph is not a tree because there is a cycle: $0 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 0$
The internal vertices are 0, 1, 3, and 4 because they each have more than one connection.
Although the graph is not a tree, vertices with degree 1 can still be identified as leaf-like vertices.
Since vertex 2 has only one connection, it is the only leaf-like vertex.
Since this is not a tree, there is no specific root unless one is assigned.



Answer:
This graph is not a tree because vertex 4 is disconnected.
Although the graph is not a tree, vertices with degree 1 can still be identified as leaf-like vertices.
Internal vertices are 0 and 2, with 1 and 3 being leaves since they have one connection.
Since the graph is not a tree, it does not have a root unless one is specifically assigned.

Part Two: Tree Properties

For Part Two, describe the key properties of trees, including edges, leaves, paths, and cycles.

2. Prove that a tree with *n* vertices has *n* − 1 **edges**.

   Answer:
   When $n = 1$, there is a single vertex and no edges
   $1 - 1 = 0$
   The statement remains true for $n = 1$.

   Assume that any tree with k vertices has exactly $k - 1$ edges.

   Consider a tree with $k + 1$ vertices
   A tree must contain at least one leaf vertex.
   Removing one leaf and its connected edge from the tree, the remaining graph still has k vertices.
   Therefore, this smaller tree has $k - 1$ edges.

   Since one edge was removed along with the leaf, adding an edge back yields: $(k - 1) + 1 = k$.
   Because the graph now has $k + 1$ vertices and k edges: $(k + 1) - 1$.
   Therefore, a tree with $k + 1$ vertices has k edges.
   Thus, a tree with n vertices has $n - 1$ edges for all positive integers n.

3. Prove that a tree with n **vertices** has at least one leaf.

   Answer:
   A leaf is a vertex with degree 1.
   A tree with n vertices must have at least one leaf because a tree is connected and has no cycles.

   Assuming a tree has no leaves for contradiction, this would mean that every vertex has at least degree 2. By starting at any vertex and following edges, we would move from vertex to vertex without getting stuck. This would mean that at some point we would have to revisit a vertex, since a tree has a finite number of vertices. This would create a cycle.

   A tree cannot have a cycle; therefore, the contradiction assumption would be false, and a tree with n vertices must have at least one leaf.

4. Explain why there is exactly one **path** between any two nodes in a tree.
   Answer:
   A tree is a connected graph with no cycles. By definition, a tree must also contain exactly $n - 1$ edges for n vertices.
   There is exactly one path between any two nodes in a tree because if there were two different paths between the same two nodes, those paths would form a cycle. This would violate the acyclicity requirement for trees.

   Additionally, having an extra path would require at least one additional edge, causing the graph to contain more than n-1 edges, which also violates a fundamental property of trees.

5. Describe an algorithm to determine if a graph contains a **cycle**.
   Answer:
   One efficient algorithm for determining whether a graph contains a cycle is Depth-First Search (DFS).

   The algorithm begins by marking all vertices as unvisited. DFS is then initiated from an unvisited vertex and recursively explores all reachable neighboring vertices within that connected component. During traversal, each visited vertex is marked to prevent redundant processing.

   For every adjacent vertex encountered:
   - If the adjacent vertex has not been visited, DFS continues from that vertex.
   - If the adjacent vertex has already been visited and is not the parent of the current vertex, then a cycle exists within the graph.

   After DFS completes for one connected component, the algorithm checks whether there are any remaining unvisited vertices. If any unvisited vertices remain, DFS is restarted from one of them to analyze the next connected component. This process continues until all vertices in the graph have been examined.

   If no previously visited non-parent vertex is encountered during traversal, the graph is acyclic.

   Because each vertex and edge is processed at most once, the overall time complexity of the algorithm is: $O(V + E)$ where V represents the number of vertices and E is the number of edges.

Part Three: Tree Traversal

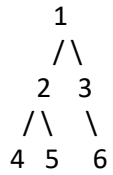
For Part Three, explain tree traversals and basic tree operations.

6. Explain **pre-order** traversal, including an example of a pre-order traversal of a binary tree.

   Answer:

   Pre-order traversal begins at the root node and then recursively traverses the left subtree followed by the right subtree.

   $Root \rightarrow Left \rightarrow Right$

   The pre-order traversal is: 1, 2, 4, 5, 3, 6

7. Explain **in-order** traversal, including an example of an in-order traversal of a binary tree.

   Answer:

   In-order traversal recursively traverses the left subtree first, then visits the root node, and finally traverses the right subtree: $Left \rightarrow Root \rightarrow Right$

   In-order traversal begins at the root and recursively moves down the left subtree to node 4. Since node 4 has no children, it is visited first. The traversal then returns to node 2, visits it, and then visits node 5. After completing the left subtree, the traversal returns to the root node 1 and visits it. The traversal then continues down the right subtree by visiting node 3 and finally node 6.

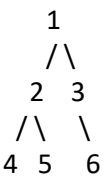
   Traversal Order: 4, 2, 5, 1, 3, 6

   If node 3 also had a left child, the traversal would first visit the left child of node 3, then node 3 itself, and finally continue to node 6.

8. Explain **post-order** traversal, including an example of a post-order traversal of a binary tree.

   Answer:

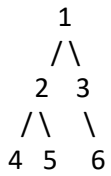
   Post-order traversal recursively traverses the left subtree first, then traverses the right subtree, and finally visits the root node: $Left \rightarrow Right \rightarrow Root$

   Post-order traversal begins at the furthest left node and recursively works upward through the tree. The traversal visits the left subtree completely before moving to the right subtree, and the root node is visited last.

   Traversal Order: 4, 5, 2, 6, 3, 1

9. Perform a **pre-order**, **in-order**, and **post-order traversal** for the provided binary tree.



Pre-Order:
1, 2, 4, 5, 3, 6
In-Order:
4, 2, 5, 1, 3, 6
Post-Order:
4, 5, 2, 6, 3, 1

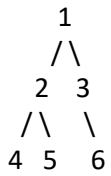
10. Find the **height** of the provided binary tree.

Answer:
The height of a binary tree is determined by the length of the longest path from the root node to a leaf node.

There are two common methods for measuring height:
- Counting the number of edges in the longest path
- Counting the number of nodes in the longest path

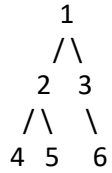
For the given tree, the longest paths are: $1 \rightarrow 2 \rightarrow 4$ $\boldsymbol{AND}$ $1 \rightarrow 2 \rightarrow 5$ $\boldsymbol{AND}$ $1 \rightarrow 3 \rightarrow 6$
Using edges, the height is 2.
Using nodes, the height is 3.

11. Find the number of **leaf nodes** of the provided binary tree.

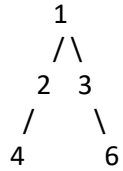


Answer:
In a rooted tree, a leaf is typically defined as a vertex with no children. In an undirected tree, leaves are also vertices with degree 1. In the given binary tree, there are 3 nodes with degree 1 (or no children), which are leaves: 4, 5, and 6. Therefore, the number of leaf nodes is 3.

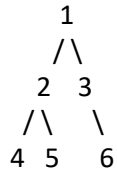
12. Delete the node with **value 5** from the provided binary tree.



Answer:



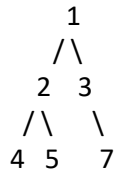
13. Insert a **new node** with value 7 as the right child of node 3 in the provided binary tree.



Answer:

This cannot be done directly because node 3 already has a right child, which is node 6.
In a binary tree, each node can only have one right child.

If node 7 must be inserted as the right child of node 3, then node 6 would need to be moved or removed first.



Part Four: Tree Applications

For Part Four, explain file systems as trees, decision trees for classification, and the properties and implementation of binary search trees.

14. Explain how a **file system** can be represented as a tree.

Answer:

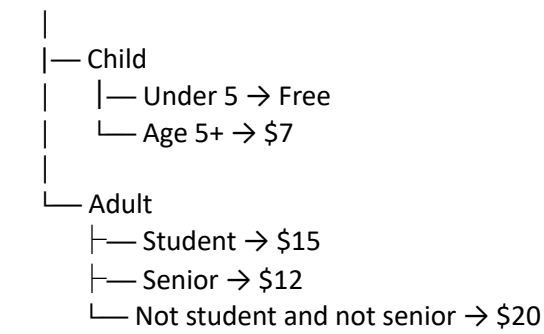
A file system can be represented as a tree because it begins with a root directory that serves as the top-level node. Within the root directory are folders (internal nodes) and files (leaf nodes). Folders may contain additional folders or files, creating a hierarchical parent-child structure that continues recursively throughout the file system.

Files are considered leaf nodes because they do not contain child elements, while folders are internal nodes because they can contain additional directories or files.

15. Identify the **root**, **internal nodes**, and **leaf nodes** in a file system tree.
    Answer:
    The **root** of the file system tree is the top-level directory or drive, such as C:\ in Windows systems. **Internal nodes** are directories or folders that contain additional folders or files. **Leaf nodes** are the individual files in the file system because they have no children or additional elements.

16. Describe how **decision trees** can be used to make decisions, including an example of a decision tree for a simple classification problem.
    Answer:
    A decision tree is a structure used to solve problems and make decisions by following a series of questions or conditions. The process begins at the root node and follows different branches depending on the answers to each question. Internal nodes represent decision points, while leaf nodes represent the final outcomes or classifications.

    Decision trees are commonly used in computer science for classification problems and can also represent nested if-else statements in programming. One example is a ticket-purchasing system in which ticket prices depend on the customer's age or status.



17. Explain the **properties** of a binary search tree.
    Answer:
    A binary search tree (BST) is a binary tree in which each node contains a unique key and follows a strict ordering property:
    - All nodes in the left subtree of a node must contain values less than the parent node.
    - All nodes in the right subtree of a node must contain values greater than the parent node.
    - Each subtree must also follow the same binary search tree properties.

    Binary search trees are especially useful for searching, inserting, and deleting data efficiently. In a balanced BST, these operations typically have $O(log\ n)$ time complexity because the tree can be traversed by repeatedly eliminating half of the remaining nodes.

18. Implement a binary search tree using **pseudocode**.

Answer:

```
BinarySearchTree:

Insert(value):
  if tree is empty:
    create root node with value

  else:
    begin at root

    while value has not been inserted:
      if value < current node:
        if left child is empty:
          insert value as left child
        else:
          move to left child

      else if value > current node:
        if right child is empty:
          insert value as right child
        else:
          move to right child

      else:
        do not insert duplicate value

Search(value):
  begin at root

  while current node exists:
    if value = current node:
      return found

    else if value < current node:
      move to left child

    else:
      move to right child

  return not found


Traversal():
  visit left subtree
  visit root
  visit right subtree
```